

编译器设计专题实验报告

实验二：词法分析扫描器实现

张凌曜 2234214014 计算机 2306

1 实验目的

用 C++ 写一个词法分析扫描器，支持多种模式：模式 1 对输入的符号串逐个分类，模式 2 对一行代码做完整的词法分析，模式 3 读取源程序文件（.src）做完整的词法分析并输出每行 Token 和统计信息。另外设计了主文法的 DFA 词法规则定义文件，让扫描器能处理代码库中任意源程序。

2 实验原理

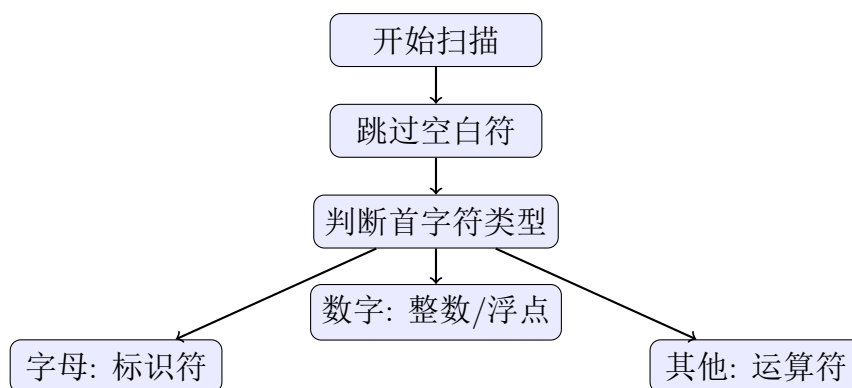
词法分析是编译的第一步，把源代码字符流拆成一个个有意义的单词（Token）。做法就是逐字符扫描，根据首字符类型进不同分支：字母开头走标识符/关键字识别，数字开头走整数或浮点数识别，其他字符处理运算符和界符。

本实验支持的词法类别包括：标识符（ID）、9 个关键字（int/float/void/if/else/while/return/input/p
整数（NUM）、浮点数（FLOAT）、算术运算符（+、-、*、/）、关系运算符（<、>、<=、>=、==、!=）、赋值符 =、逻辑运算符（&&、||）、括号和分号等。

3 实验过程

3.1 识别流程

扫描器逐字符读入，先跳过空白符，再看首字符：



3.2 实现要点

浮点数判断比较麻烦，得处理三种格式：普通小数（123.45）、小数点开头（.45）、科学计数法（1e-3）。我的做法是先找有没有‘e’或‘E’，有的话分离基数和指数分别验证；没有的话就数小数点个数，恰好 1 个就是浮点数。从 DFA 的角度看，这个过程本质上也是状态转移：读入数字进入“整数”状态，遇到小数点转移到“小数部分”状态，遇到 e/E 再转移到“指数”状态（指数部分还可能读入正负号），最终停在哪个接受状态就决定是 NUM 还是 FLOAT。

双字符运算符（<=、>=、==、!=、+=、++）必须在单字符之前判断，不然“<=”会被拆成“<”和“=”两个 Token。同理，浮点数判断得在整数判断之前，不然“3.14”会被拆成“3”和“.14”。

关键字识别比较简单，先当普通标识符读出来，再查关键字表看是不是关键字就行。关键字表用的是 `std::set<string>` 实现，查找复杂度 $O(\log n)$ ，关键字数量少所以效率足够。

3.3 DFA 词法规则文件

为支持主文法，设计了 `main_grammar.dfa` 文件，用 DFA 五元组形式定义了每种 Token 的状态转移规则：标识符/关键字 DFA（字母开头，字母或数字继续）、整数 DFA（数字序列）、浮点数 DFA（含小数点和科学计数法的 7 个状态）、比较运算符 DFA（<、>、=、! 的双字符转移）等。共 8 类 Token DFA，覆盖代码库中所有 .src 文件的词法需求。

另外，扫描器还处理了 // 行注释：遇到连续两个斜杠就跳过该行剩余内容。

4 测试结果

4.1 模式 1 测试

输入：5 / id if 485 841.6541 www

输入	输出
id	ID
if	IF
485	NUM
841.6541	FLOAT
www	ID

```

PS C:\Users\corey\Desktop\编译实验\第二次实验> .\scanner.exe
=====
  Lexer + DFA Integration
=====

1. Token classification (input n tokens)
2. Code line analysis (input one line)
3. DFA string recognition (load .dfa file)
0. Exit
> 1
Number of tokens: 5
id if 485 841.6541 www
id -> ID
if -> IF
485 -> NUM
841.6541 -> FLOAT
www -> ID

```

4.2 模式 2 测试

输入代码: `while(true) {int a=0;}`

输出: WHILE, LPAR, ID, RPAR, LBR, INT, ID, ASG, NUM, SEMI, RBR

其中 `true` 没作为关键字处理(按标识符输出 ID), `int` 正确识别为 INT。这里 `true` 输出 ID 是符合预期的——本实验定义的关键字集合只包含 9 个(`int/float/void/if/else/while/return/in` 没有把布尔字面量 `true` 纳进去, 所以词法分析器把它当作合法标识符处理。

```

> 2
Input code: while(true) {int a=0;}
Tokens:
(WHILE, while) -> (LPAR, () -> (ID, true) -> (RPAR, )) -> (LBR, {) -> (INT, in
t) -> (ID, a) -> (ASG, =) -> (NUM, 0) -> (SEMI, ;) -> (RBR, })

```

5 选做：整合 DFA

在词法分析器的基础上加入了菜单系统, 程序启动后可以选择四种功能: 符号串分类(模式 1)、代码行分析(模式 2)、源文件扫描(模式 3)、DFA 字符串识别(模式 4)。

模式 3 可以读取任意 .src 文件，逐行扫描并输出每个 Token 的类型、值和行号，最后给出统计摘要。经测试，代码库中 33 个 .src 文件均可正确解析。

模式 4（DFA 整合）的思路：读取 .dfa 文件加载五元组，用转移表查表做字符串识别，输出状态路径和接受/拒绝结果。

测试：加载 dfa_in1.dfa，输入字符串 ab，程序输出路径 $1 \rightarrow 2 \rightarrow 3$ ，判定为接受。

```
> 3
DFA file path: dfa_in1.dfa
DFA loaded! Alphabet: {a, b}, Accept: {3}
Input string: ab
Path: 1 -> 2 -> 3
Result: ACCEPT
```

6 实验总结

这个实验主要踩了两个坑：一个是判断顺序很重要，浮点数必须在整数前面判断，双字符运算符必须在单字符前面判断；另一个是科学计数法的指数部分可能带正负号（ $1e-3$ ），不能一看到减号就当成减法运算符。

选做部分把 DFA 整合进来后，可以直观地看到词法分析和 DFA 模拟的关系——词法分析器内部其实就是多个 DFA 在并行工作，每个 Token 类别对应一个 DFA。

词法分析器虽然只是编译的第一步，但 Token 流的质量直接影响后面语法分析，所以边界情况处理还是比较重要的。

A 附录：代码仓库

本实验的全部源码、报告及 Web 演示平台已托管至 GitHub：

<https://github.com/yaossss123/compilation-web>